a cyberwitchery talk

type systems you might not know

(but will love)

veit heller
cyberwitchery lab

2026

section 01

01 beyond static vs dynamic

the menu you were handed

> static vs dynamic.
> strong vs weak.
> most of us stopped there.

the usual two axes

static vs dynamic is about when checks run.

strong vs weak barely has a consistent meaning.

neither is the interesting axis.

a better question

i think it's more useful to ask
what a type can say about your program.

what can a type say?

> “this is an integer.”
> “this is a list of strings.”
> “this integer is never zero.”
> “this list is non-empty, so `head` is safe.”
> “this socket has already been closed.”
> “this function touches the disk.”
> “send a request, then receive exactly one reply.”

the greyed-out two, you say every day. the rest is today's talk.

the plan

> five things a type can say, **`from familiar to strange`**.
> the first is probably familiar. the last will not be.
> a roadmap, so you leave with somewhere to go.

one trip to the terminal, to build a type checker ourselves.

section 02

02 five things a type can say

ownership

“this value has exactly one owner.”

use it once, then move it or borrow it. rust's borrow checker is one of these, whether or not you have met it.

can’t close it twice

```
let c = Conn::open("socket-7");
c.close();
c.close();
```

```
error[E0382]: use of moved value: `c`
note: `close` takes ownership of the receiver `self`,
which moves `c`
```

close consumes the handle, so a second use can't typecheck.

that’s an “affine” type

- **`linear`**: use a value *exactly* once.
- **`affine`**: use it *at most* once.
- **`rust`**: affine, unless a type is `Copy` (values safe to duplicate).

a large class of use-after-free, double-free, and data-race bugs becomes a compile error. `Send` and `Sync` carry thread-safety in the type.

→ live

let's write a type checker.

a real type system

55 lines, no solver

that is a working refinement type checker, in plain interval arithmetic. real systems (liquid haskell, f*) hand the subtyping to an smt solver.

effect systems

```
readFile : Path →{IO} Text
```

unison

“this function touches the disk.”

in koka, eff, and unison, the effect rides in the type next to the return value.

session types

```
!Register ; ?Job ; !Result ; Close
```

freest

“the worker can’t get a job before it registers.”

two concurrent processes talking over one channel. in freest the order is part of the channel's type, so sending them out of order won't compile.

dependent types

```
head : Vec A (suc n) → A
```

agda

“this list is non-empty, so `head` is always safe.”

types can depend on *values*. `Vec A n` carries its length, so `head []` does not typecheck.

proofs as programs

```
_++_ : Vec A m → Vec A n → Vec A (m + n)
```

agda

“concatenation adds the lengths.”

the signature is the spec, and the two-line body is a proof of it, checked at compile time.

five claims, side by side

	the bug, at runtime	**caught at compile time**
ownership	use after close, double free	moved, so reuse won't compile
refinement	divide by zero	argument must be non-zero
effects	a hidden disk write	the effect is in the signature
session	a message out of order	wrong message won't compile
dependent	`head []`, off by one	the length lives in the type

section 03

03 a roadmap

pick an axis, not a language

- resource safety $\rightarrow$ rust.
- proof $\rightarrow$ agda, idris, lean.
- effects $\rightarrow$ koka, unison.
- contracts $\rightarrow$ liquid haskell, f*.

you don't need a phd to use any of these in anger.

resources

- *types and programming languages*, pierce.
- *programming language foundations in agda*.
- *the little typer*, friedman & christiansen.

none of them will melt your brain.

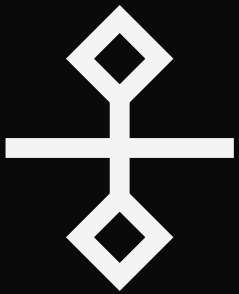
the takeaway

a type system is a language
for saying what's true.

pick one that lets you say
what you mean.

thanks.

questions?



veit heller veit@veitheller.de

slides: github.com/hellerve/talks